

Proyecto 2 — Resultados: Clasificación de Acciones en Video de la Bundesliga (Pase, Disputa, Saque de Banda y Fondo)

Carlos Nicolás Concuá Trujillo (23197)

Jorge Luis Felipe Aguilar Portillo (23195)

Fernando José Rueda Rodas (23748)

Fernando Andree Hernández Martínez (23645)

Departamento de Ciencias de la Computación, Universidad del Valle de Guatemala

CC3084: Data Science

15 de octubre de 2026

Repositorio: https://github.com/nicoCT04/Proyecto2-DataScience. Aplicación: streamlit run app/app.py.

1. Introducción y objetivos

1.1 Introducción

En la fase de análisis exploratorio se caracterizaron los 200 clips de video de la Bundesliga y se confirmó que las señales visuales necesarias para reconocer acciones con balón están presentes: la cancha se aísla por color, los jugadores son detectables y el movimiento entre frames marca los momentos de acción. Esta fase de **resultados** da el siguiente paso: construir y evaluar modelos que clasifiquen automáticamente la acción de cada jugada a partir de esas señales.

El problema se plantea como una **clasificación de ventanas de video (subclips)** en cuatro categorías: pase (*play*), disputa (*challenge*), saque de banda (*throwin*) y fondo sin acción (*background*). Para cada subclip se extrajeron variables agregadas de movimiento, color, bordes, cámara, jugadores y balón, y a partir de ellas se entrenaron dos familias de modelos: modelos **tabulares** sobre las variables resumidas (regresión logística, Random Forest y XGBoost) y un modelo **temporal** (GRU) sobre la secuencia de embeddings de los frames. La evaluación se hace sobre un **partido completo no visto** durante el entrenamiento, para medir la generalización de forma realista.

1.2 Objetivos

Objetivo general. Desarrollar y comparar modelos de clasificación que identifiquen la acción (pase, disputa, saque de banda o fondo) en subclips de video de la Bundesliga, evaluando su capacidad de generalizar a un partido no visto.

Objetivos específicos.

1. Entrenar modelos tabulares (regresión logística, Random Forest y XGBoost) sobre las variables agregadas de cada subclip, ajustando sus hiperparámetros mediante validación cruzada por partido (GroupKFold sobre match_id) con métrica F1 macro.
2. Entrenar un modelo temporal (GRU) sobre la secuencia de embeddings de ResNet18 de los frames y compararlo con los modelos tabulares.
3. Evaluar todos los modelos en un partido de prueba retenido, reportando F1 macro, F1 por clase y matrices de confusión, e interpretar los errores.
4. Integrar el mejor modelo en una aplicación que permita explorar los datos y predecir la acción sobre video nuevo.

2. Marco teórico

2.1 Reconocimiento de acciones en fútbol

El análisis automático de fútbol comprende tareas distintas: detectar jugadores y balón, seguir sus trayectorias, reconocer eventos e interpretar el juego. Akan y Varlı (2023) revisan estas aplicaciones y muestran que el reconocimiento de eventos forma parte de un problema más amplio de comprensión del video. En este proyecto nos concentramos en clasificar ventanas cortas en play, challenge, throwin y background. La categoría play se utiliza como aproximación al pase según las anotaciones disponibles; no determina por sí sola si un pase fue exitoso ni quién lo realizó.

Una imagen aislada puede contener jugadores y cancha sin mostrar la acción que ocurre. La secuencia aporta cambios de posición y movimiento, pero también incluye oclusiones, variaciones de escala y cambios de iluminación o punto de vista. Estos son problemas habituales del reconocimiento de actividades descritos por Vrigkas et al. (2015). En una transmisión

deportiva, además, el movimiento de cámara puede confundirse con el desplazamiento de los jugadores o del balón.

Clasificar subclips ya delimitados no equivale a localizar automáticamente todos los pases en un partido continuo. Esta última tarea requiere proponer ventanas o instantes candidatos y evaluar su localización temporal. Por ello, el alcance de nuestros clasificadores es reconocer la clase de cada ventana suministrada.

2.2 Representación visual y preprocesamiento

Las representaciones de video pueden combinar apariencia y dinámica temporal (Vrigkas et al., 2015). En el repositorio se siguen dos rutas. La primera resume cada ventana mediante variables de color, bordes, flujo óptico, movimiento de cámara y detecciones de jugadores y balón. El flujo óptico aproxima desplazamientos entre imágenes consecutivas; no identifica por sí mismo una acción. Las detecciones permiten resumir posiciones y cantidades, pero sus errores se trasladan a las variables calculadas.

La segunda ruta utiliza una red convolucional para representar cada cuadro. ResNet introduce conexiones residuales que suman la entrada de un bloque a su transformación, facilitando el entrenamiento de redes profundas (He et al., 2016). Aquí se reutiliza ResNet18 preentrenada en ImageNet, sin su capa clasificadora y con pesos congelados. Cada cuadro queda representado por un vector de 512 componentes (*embedding*); este vector describe apariencia, no una etiqueta de pase.

El preprocesamiento existente toma 16 cuadros equiespaciados por subclip, recorta las franjas superior e inferior y prepara imágenes RGB de 224 × 224 con la normalización de ImageNet. La entrada temporal conserva así una matriz de 16 × 512, en orden cronológico. Reutilizar los embeddings evita entrenar toda la CNN con los 400 ejemplos de entrenamiento,

aunque una representación aprendida en imágenes generales puede no capturar bien detalles pequeños como el balón.

2.3 Modelo temporal GRU

Una red recurrente actualiza un estado oculto a partir del cuadro actual y del estado anterior. La unidad recurrente con compuertas, GRU, utiliza una compuerta de actualización para regular cuánto estado conserva y una de reinicio para modular la información previa al calcular el estado candidato (Cho et al., 2014). Esto permite modelar dependencias entre cuadros, en lugar de reducir toda la ventana a un promedio.

Ullah et al. (2018) estudiaron una combinación de características CNN y una LSTM bidireccional para reconocer acciones en video. Su trabajo respalda la estrategia de separar representación espacial y modelado temporal, pero no demuestra que una GRU sea superior ni garantiza resultados equivalentes en fútbol. Elegimos una GRU unidireccional de una capa como alternativa temporal compacta para este conjunto pequeño; su utilidad debe comprobarse experimentalmente.

La implementación recibe los embeddings, los estandariza con estadísticas del entrenamiento y utiliza el último estado oculto para clasificar el subclip. Una capa *dropout* de 0.3 precede a la salida lineal de cuatro clases. El entrenamiento minimiza entropía cruzada con Adam y recorte del gradiente. Se comparan 32 y 64 unidades ocultas y 20 y 40 épocas; la selección depende del F1 macro de validación, no del error sobre el partido de prueba. Los pesos y la normalización se guardan juntos para reproducir la inferencia.

2.4 Modelos tabulares de comparación

La regresión logística multinomial funciona como referencia lineal: transforma las variables estandarizadas en probabilidades de clase. Random Forest combina árboles entrenados

con aleatoriedad en los datos y las variables, lo que permite representar relaciones no lineales (Breiman, 2001). El *gradient boosting* construye un modelo aditivo de forma secuencial (Friedman, 2001); XGBoost desarrolla esta familia con regularización y optimizaciones de cálculo (Chen y Guestrin, 2016).

Estas alternativas permiten contrastar una referencia lineal, ensambles de árboles y una representación temporal. Sin embargo, los modelos tabulares reciben variables agregadas y la GRU recibe embeddings CNN. La comparación mide sistemas completos, no exclusivamente el efecto del algoritmo: no permite atribuir cualquier diferencia de desempeño solo a la arquitectura recurrente.

2.5 Validación y límites de la comparación

Los subclips de un mismo partido comparten condiciones visuales. Separarlos al azar podría favorecer resultados optimistas. Por eso usamos cinco pliegues agrupados por match_id dentro de los siete partidos de entrenamiento. En la GRU, la media y la desviación de cada componente se calculan únicamente con el entrenamiento de cada pliegue; después de seleccionar parámetros, se reajusta sobre los 400 subclips.

El F1 de cada clase combina precisión y exhaustividad mediante su media armónica; el F1 macro promedia las cuatro clases con igual peso. Así, la clase menos frecuente no queda oculta por las demás. La matriz de confusión y el F1 por clase complementan esta medida al mostrar qué acciones se confunden.

El conjunto denominado valid en los archivos corresponde al partido retenido para la prueba final y no interviene en la selección de la GRU. El mejor F1 de la búsqueda es una medida de selección, no una estimación independiente. La evaluación final debe realizarse por separado y sus conclusiones quedan limitadas por disponer de un único partido de prueba. La

revisión se apoya en artículos de revista y se complementa con los trabajos originales de congreso sobre GRU, ResNet y XGBoost.

3. Metodología

3.1 Datos y construcción de los subclips

A partir de las anotaciones de train.csv se recortaron **762 subclips** centrados en los eventos del juego, etiquetados en cuatro clases: background (200), challenge (200), play (200) y throwin (162). Cada subclip conserva su partido de origen (match_id), lo que permite separar entrenamiento y prueba por partido.

Los datos se dividen en dos conjuntos **por partido**, nunca mezclando frames de un mismo encuentro entre entrenamiento y prueba:

- **Entrenamiento:** 400 subclips de 7 partidos.
- **Prueba (partido no visto):** 362 subclips del partido 3c993bd2.

3.2 Variables

De cada subclip se calcularon **48 variables agregadas** que resumen la ventana completa, agrupadas en:

- **Color e iluminación:** brillo, contraste, saturación y cobertura de cancha (green_ratio), con su media, desviación y extremos.
- **Detalle y cortes:** densidad de bordes y detección de cortes de cámara (shot_cut).
- **Movimiento:** movimiento global entre frames, movimiento de cámara, flujo óptico de los jugadores, coherencia del flujo y posición del pico de movimiento.
- **Jugadores:** número medio de jugadores detectados (YOLOv8) y su dispersión espacial.
- **Balón:** visibilidad, velocidad, altura y distancia al jugador más cercano.

No se encontraron valores faltantes en la matriz de variables. La duración del subclip (duration_s) y el contexto previo al evento (lead_s) se excluyeron de los modelos: dependen de cómo se recortó cada clip y no de la acción (ver sección 4.2).

3.3 Modelos

Se entrenaron dos familias de modelos complementarias:

- **Modelos tabulares** sobre las 48 variables agregadas: **regresión logística** (con estandarización), **Random Forest** y **XGBoost**. Resuelven el problema con la información resumida de toda la ventana.
- **Modelo temporal (GRU)** sobre la secuencia de embeddings de ResNet18 de los frames, que aprovecha la dinámica cuadro a cuadro (ver sección de marco teórico).

3.4 Ajuste de hiperparámetros y validación

El ajuste de los modelos tabulares se hizo con **búsqueda en malla** y **validación cruzada por partido** (GroupKFold sobre match_id, 5 pliegues), de modo que en cada pliegue se validan partidos distintos a los de entrenamiento. La métrica de selección es el **F1 macro**, que pondera por igual las cuatro clases pese al leve desbalance de throwin. Con los mejores hiperparámetros, cada modelo se reentrena sobre todo el conjunto de entrenamiento y se guarda para su evaluación e integración en la aplicación.

3.5 Protocolo de evaluación

El desempeño final se mide sobre el **partido de prueba retenido**, reportando F1 macro, F1 por clase, matrices de confusión y curvas de precisión-recuerdo. Evaluar sobre un partido completo no visto es deliberadamente exigente: refleja el uso real del sistema —analizar un encuentro nuevo— y evita el optimismo de mezclar frames de un mismo partido entre entrenamiento y prueba.

3.6 Herramientas y recursos de cómputo

- **Recursos de cómputo.** Todo el proyecto se ejecutó en una computadora portátil Apple M4 Pro con 24 GB de memoria. YOLOv8 y ResNet18 corren en la GPU integrada mediante Metal (MPS) y los modelos se entrenan en CPU. No se necesitó GPU dedicada porque la CNN se usa congelada y los modelos de clasificación son pequeños; extraer las variables de una ventana tarda entre 0.4 y 2 s.
- **Lenguaje y bibliotecas.** Python 3.11 con OpenCV (lectura de video, flujo óptico de Farnebäck, color y bordes), Ultralytics YOLOv8 (jugadores y balón), PyTorch y torchvision (ResNet18 y GRU), scikit-learn (regresión logística, Random Forest, validación cruzada y métricas), XGBoost, pandas y NumPy (datos), Matplotlib (figuras del informe), Streamlit y Plotly (aplicación y gráficas interactivas) y python-docx (armado del informe).
- **Versionado.** El código, los modelos entrenados y los resultados se versionan en GitHub; los videos se descargan con scripts desde Hugging Face porque pesan varios GB.

4. Resultados y análisis de resultados

4.1 Características del conjunto de datos original

Los datos de esta fase son **subclips etiquetados** derivados de los videos anotados de la competencia *DFL – Bundesliga Data Shootout*, publicados en Hugging Face como ManuD/dfl_clips. Cada subclip es un archivo .mp4 a 1920 × 1080 y 25 fps cuyo nombre codifica el video, el milisegundo del evento y el del evento siguiente (<partido>_<tiempo>_<ms evento>_<ms siguiente>.mp4). El conjunto tiene **762 subclips de 8 partidos** (Tabla 1).

Tabla 1

Subclips por conjunto y clase.

Conjunto	Partidos	Fondo	Disputa	Pase	Saque de banda	Total

Entrenamiento	7	100	100	100	100	400
Prueba (3c993bd2)	1	100	100	100	62	362

Antes de usarlos se verificó la calidad de las etiquetas: los 562 subclips con acción coinciden con un evento del mismo tipo en train.csv a menos de 0.5 s (100 %). Los subclips varían mucho de duración (de 0.3 s a 121.5 s; mediana de 160 cuadros) y esa duración depende de la clase: los de fondo duran en promedio 11.0 s en entrenamiento y los de acción entre 5.3 y 6.4 s. El contexto disponible antes del evento tiene una mediana de 5.3 s, pero el 13.9 % de los subclips tiene menos de 1.5 s.

4.2 Limpieza y preprocesamiento

El preprocesamiento es el mismo para el entrenamiento y para la aplicación (src/preprocessing.py y src/features.py), de modo que un video nuevo pasa exactamente por los mismos pasos:

5. **Recuperación del contexto.** Los subclips se cortaron sin recodificar y una *edit list* oculta los cuadros previos al evento; se lee el archivo ignorándola para recuperar hasta 10 s de contexto y se ubica el evento a partir de los milisegundos del nombre.
6. **Ventana fija.** De cada subclip se toma una ventana de 1.62 s (1.5 s antes del evento y 0.12 s después) muestreada en **16 cuadros equiespaciados** (~10 fps). Cuando falta contexto se repite el primer cuadro en lugar de desplazar la ventana, para que el evento caiga siempre en la misma posición.
7. **Recorte de superposiciones.** Se elimina el 10 % superior (marcador y marca de agua) y el 4 % inferior de cada cuadro, hallazgo del análisis exploratorio.

8. **Redimensionado.** 320 × 180 para las variables clásicas y el flujo óptico, 224 × 224 con normalización de ImageNet para ResNet18 y 1280 px de ancho para YOLOv8 (el balón es muy pequeño a menor resolución).
9. **Extracción de variables.** 50 variables agregadas por ventana (color, bordes, movimiento, flujo óptico de Farnebäck separando el paneo de cámara del movimiento propio, jugadores y balón con YOLOv8n) y una secuencia de 16 × 512 *embeddings* de ResNet18 (He et al., 2016). La matriz no tiene valores faltantes: cuando YOLO no ve el balón, sus variables toman un valor neutro documentado (posición 0.5, velocidad 0).

Corrección de una fuga de información. Al evaluar los primeros modelos se encontró que la variable más importante del Random Forest era duration_s, la duración del subclip. Como los recortes de fondo son casi el doble de largos, el modelo aprendía *cómo se cortó el clip* y no la acción; además, esa variable no existe para un video nuevo en la aplicación. Se excluyeron duration_s y lead_s (contexto previo al evento) y se reentrenaron los tres modelos tabulares con **48 variables**. El F1 macro de XGBoost en el partido de prueba bajó de 0.381 a 0.347: esa diferencia era optimismo, no capacidad de reconocer acciones.

4.3 Análisis exploratorio

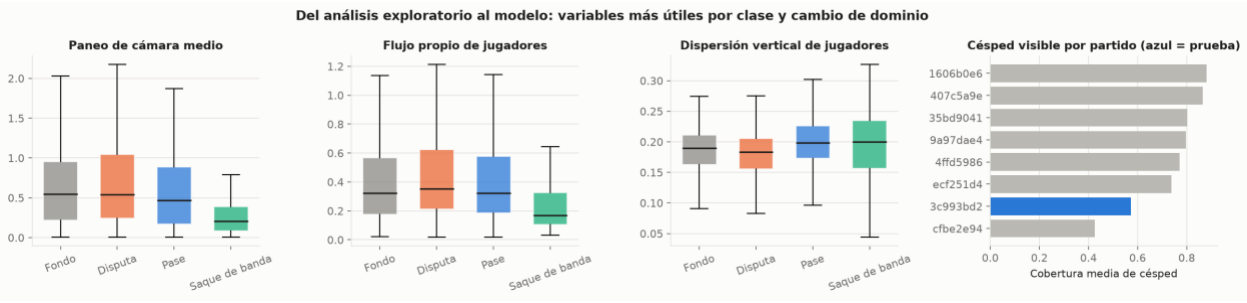
El análisis exploratorio de la fase anterior, sobre 200 clips de 20 partidos, había establecido que la cancha se aísla por color pero su cobertura cambia mucho entre partidos (mediana de 0.42 a 0.81), que YOLOv8 detecta en promedio 11.9 jugadores por cuadro y que el movimiento entre cuadros separados por ~3 s refleja cambios de plano, no el balón. Por eso en esta fase se trabaja con cuadros sucesivos (~10 fps) y se separa el movimiento de cámara del movimiento propio.

Sobre los 762 subclips, la Figura 1 resume los hallazgos que más influyen en los modelos:

- El **saque de banda** es la acción más quieta: mediana de paneo de cámara de 0.20 px por cuadro frente a 0.47–0.55 en las demás clases, y la mitad de flujo propio de jugadores (0.17 frente a 0.32–0.35).
- **Pase, disputa y fondo se superponen** en casi todas las variables. Sus distribuciones de movimiento, color y jugadores tienen medianas casi iguales; ninguna pareja de variables las separa (se puede comprobar en el diagrama de dispersión de la aplicación).
- **Cada partido se ve distinto.** La cobertura media de césped va de 0.43 a 0.88 según el partido; el de prueba (0.57) muestra menos cancha que 6 de los 7 partidos de entrenamiento. Este cambio de dominio explica buena parte de la brecha entre el ajuste y la prueba (Torralba y Efros, 2011).

Figura 1

Variables más útiles por clase (paneo de cámara, flujo propio y dispersión vertical de jugadores) y cobertura de césped por partido.



4.4 Ajuste de hiperparámetros

Todos los modelos se ajustaron solo con los 7 partidos de entrenamiento, mediante validación cruzada agrupada por partido (GroupKFold, 5 pliegues) y F1 macro como métrica. La Tabla 2 muestra la malla explorada y la combinación elegida.

Tabla 2

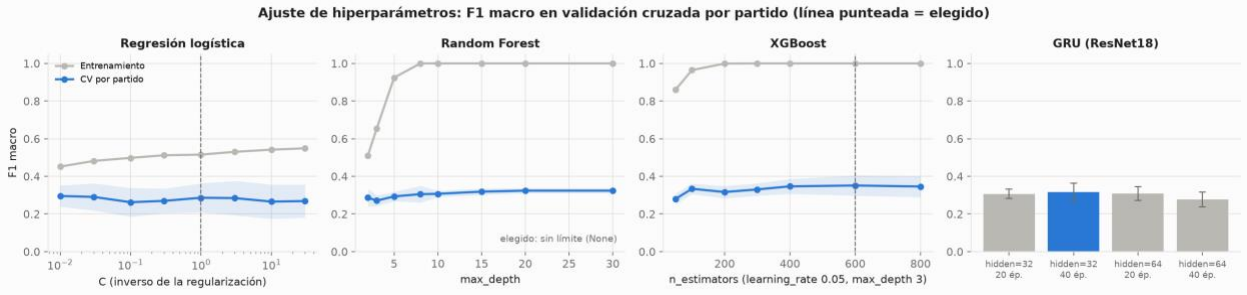
Ajuste de hiperparámetros con validación cruzada por partido.

Modelo	Malla explorada	Elegido	F1 macro CV
Regresión logística	C ∈ {0.1, 1, 10}; class_weight ∈ {None, balanced}	C = 1, sin pesos	0.286 ± 0.077
Random Forest	árboles ∈ {300, 600}; profundidad ∈ {None, 10, 20}; hojas mín. ∈ {1, 2, 4}	300 árboles, sin límite, hoja 1	0.324 ± 0.014
XGBoost	árboles ∈ {300, 600}; profundidad ∈ {3, 5}; tasa ∈ {0.05, 0.1}	600 árboles, profundidad 3, tasa 0.05	0.351 ± 0.054
GRU	unidades ocultas ∈ {32, 64}; épocas ∈ {20, 40}	32 unidades, 40 épocas	0.316 ± 0.048

Para entender la sensibilidad de cada modelo se trazaron curvas de validación del hiperparámetro principal (Figura 2). Los dos ensambles de árboles alcanzan F1 = 1.0 en entrenamiento desde profundidad 8 o desde 200 árboles, mientras que la validación por partido se estanca entre 0.30 y 0.35. En la regresión logística, C apenas cambia el resultado (0.26–0.30). Es decir, **el límite no es la capacidad del modelo**: las variables cambian de un partido a otro y más complejidad solo memoriza el entrenamiento. La regularización de XGBoost (tasa 0.05 y árboles poco profundos) es lo que le da la mejor validación.

Figura 2

Curvas de validación por partido del hiperparámetro principal de cada modelo y candidatos de la GRU (línea punteada = valor elegido).



4.5 Comparación de los algoritmos

La evaluación final se hizo **una sola vez** sobre el partido de prueba 3c993bd2, que no intervino en ningún ajuste (src/evaluate.py). La Tabla 3 reúne las métricas; los tiempos se midieron en un Apple M4 Pro y excluyen la extracción de variables, que cuesta entre 0.4 y 2 s por ventana y domina el tiempo total.

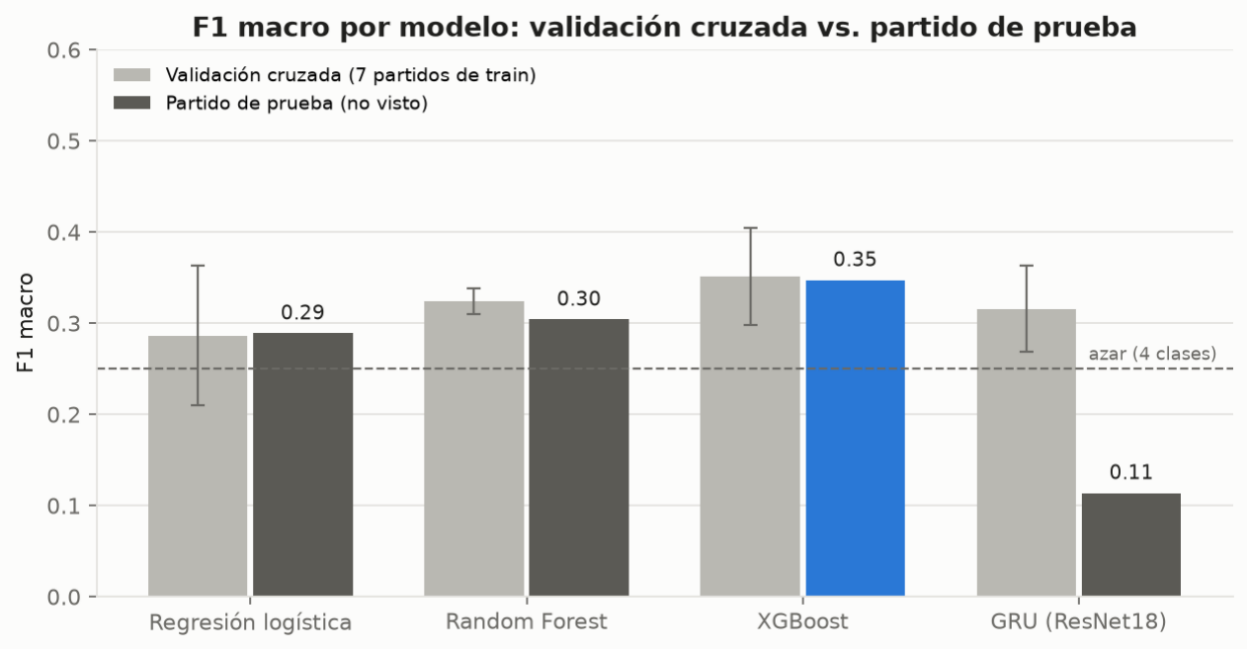
Tabla 3

Métricas en el partido de prueba (en negrita, el mejor valor de cada columna).

Modelo	F1 macro	Exactitud	Exact. balanceada	Precisión macro	AP macro	F1 macro CV	Ajuste	Inferencia por subclip
Regresión logística	0.289	0.293	0.302	0.316	0.292	0.286	~0.01 s	~0.5 µs
Random Forest	0.304	**0.367**	**0.377**	**0.494**	**0.368**	0.324	~0.2 s	~70 µs
XGBoost	**0.347**	0.362	0.372	0.355	0.359	**0.351**	~0.9 s	~7 µs
GRU (ResNet18)	0.114	0.180	0.247	0.291	0.279	0.316	~1.2 s	~6 µs

Figura 3

F1 macro de cada modelo en validación cruzada (gris claro) y en el partido de prueba. La línea punteada marca el azar con cuatro clases.



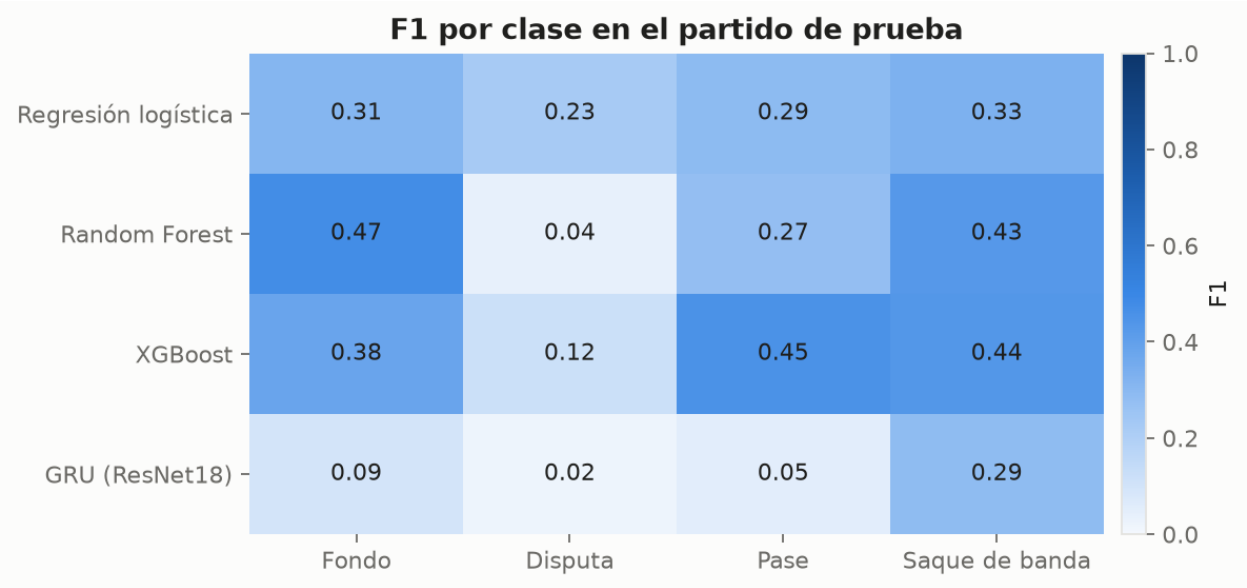
XGBoost es el mejor modelo por F1 macro (0.347) y el único que mantiene en el partido nuevo casi el mismo desempeño que en validación cruzada (0.351), lo que indica que su estimación de ajuste era honesta. El Random Forest tiene mejor exactitud y precisión, pero porque se inclina hacia la clase fondo: la predice en el 70 % de los casos y casi nunca predice disputa. La regresión logística, lineal, queda apenas sobre el azar (0.25).

La **GRU no generaliza**: obtuvo 0.316 en validación cruzada pero 0.114 en prueba, porque predice saque de banda en 322 de los 362 subclips (Figura 5). Sus *embeddings* de ResNet18 describen apariencia general (colores, encuadre, público) y el partido de prueba tiene una apariencia distinta; con solo 400 secuencias de entrenamiento la red no aprende a separar la acción de la apariencia del partido. Como advierte el marco teórico, la comparación es entre

sistemas completos: la caída no prueba que una arquitectura recurrente sea peor, sino que estos *embeddings* congelados no transfieren.

Figura 4

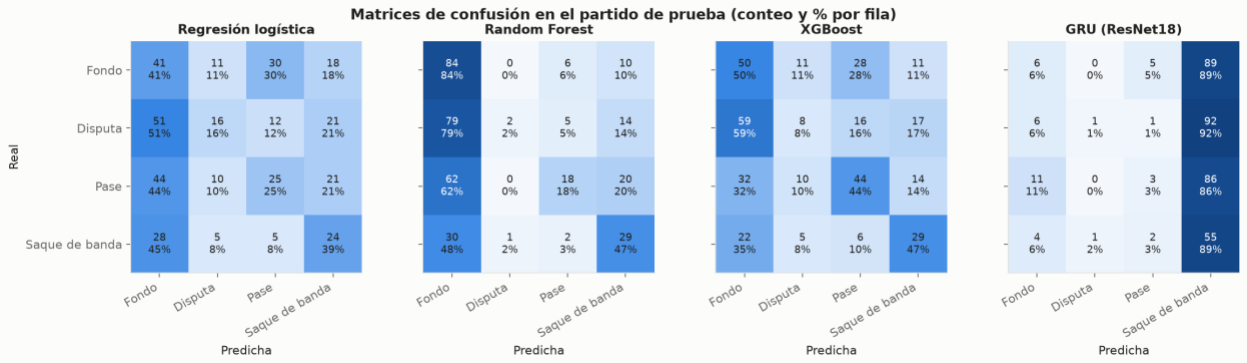
F1 por clase en el partido de prueba.



El desempeño por clase (Figura 4) confirma lo que anticipaba el análisis exploratorio. En XGBoost, el **pase** (F1 0.45) y el **saque de banda** (0.44) son las clases mejor reconocidas: el saque por ser la acción más quieta y el pase por la combinación de flujo y dispersión de jugadores. La **disputa** es casi invisible para todos los modelos (F1 entre 0.02 y 0.23): la matriz de confusión (Figura 5) muestra que XGBoost la confunde con fondo en el 59 % de los casos y el Random Forest en el 79 %. En 1.6 s, una disputa no tiene un patrón de movimiento distinto al del juego sin acción.

Figura 5

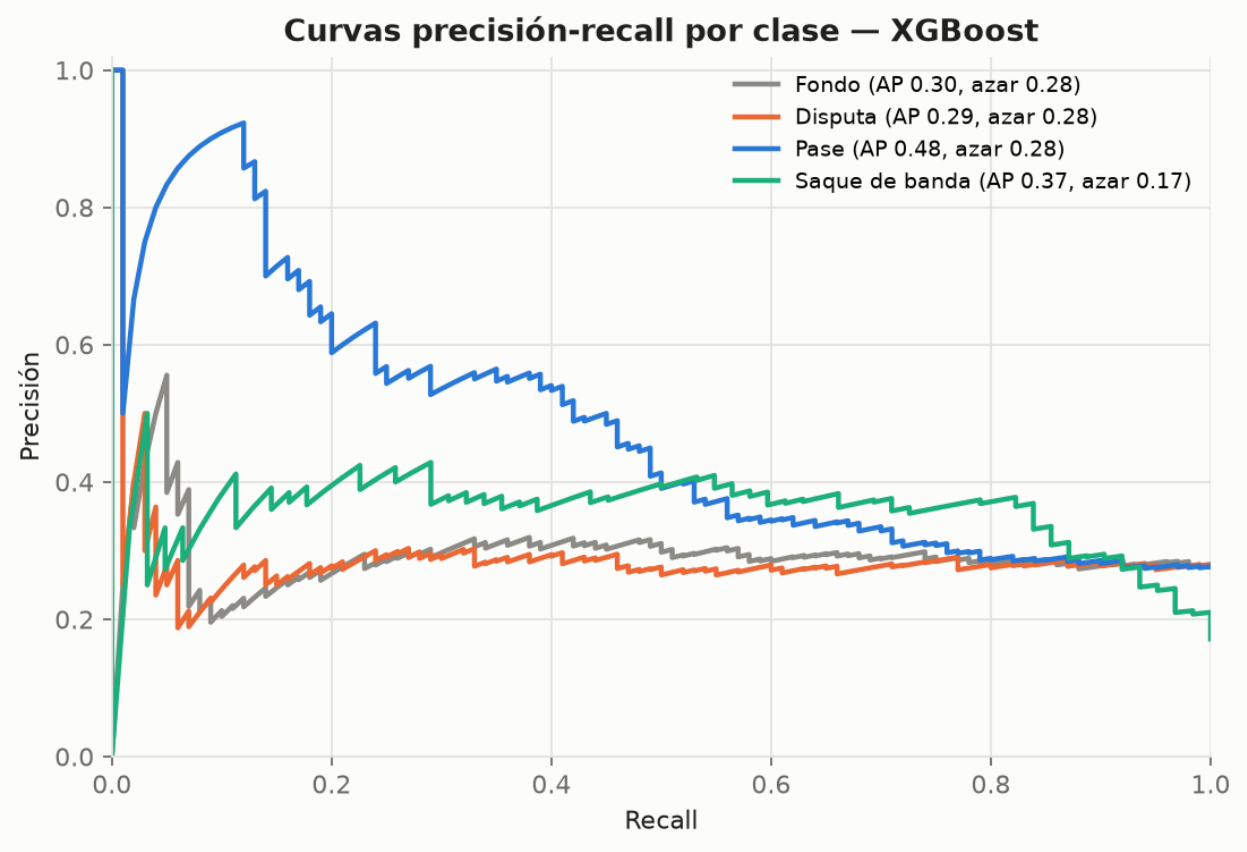
Matrices de confusión en el partido de prueba (conteo y porcentaje por fila).



Las curvas de precisión-recall (Figura 6) son más informativas que la exactitud cuando importa una clase concreta (Saito y Rehmsmeier, 2015). Para el pase, XGBoost alcanza una precisión cercana a 0.9 en el 10 % de recall: cuando el modelo está muy seguro, casi siempre acierta. Su precisión promedio (AP 0.48) casi duplica la prevalencia (0.28), y la del saque de banda (0.37) duplica la suya (0.17); en cambio, fondo y disputa quedan apenas sobre el azar. El modelo sirve para **encontrar los pases más claros**, no para detectarlos todos.

Figura 6

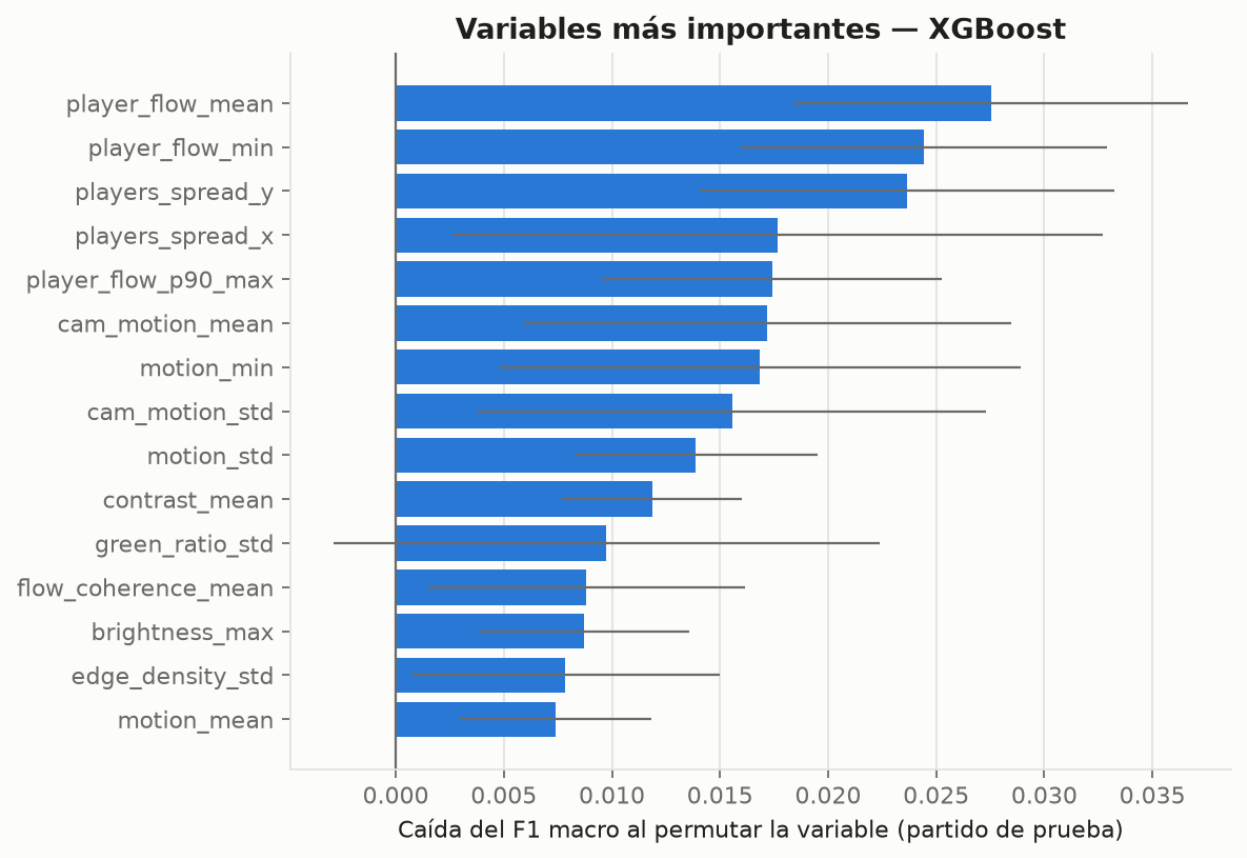
Curvas de precisión-recall por clase de XGBoost (AP = precisión promedio; azar = prevalencia de la clase).



La importancia por permutación (Breiman, 2001), medida como la caída del F1 macro al barajar cada variable en el partido de prueba, indica que XGBoost se apoya sobre todo en el **flujo propio de jugadores** (player_flow_mean, player_flow_min, player_flow_p90_max), la **dispersión de jugadores** detectados por YOLO y el **paneo de cámara** (Figura 7). Son justamente las variables que el análisis exploratorio señaló como diferentes entre clases y las que se diseñaron para separar el movimiento de la cámara del de los jugadores. Las variables de color e iluminación, que más cambian entre partidos, aportan poco.

Figura 7

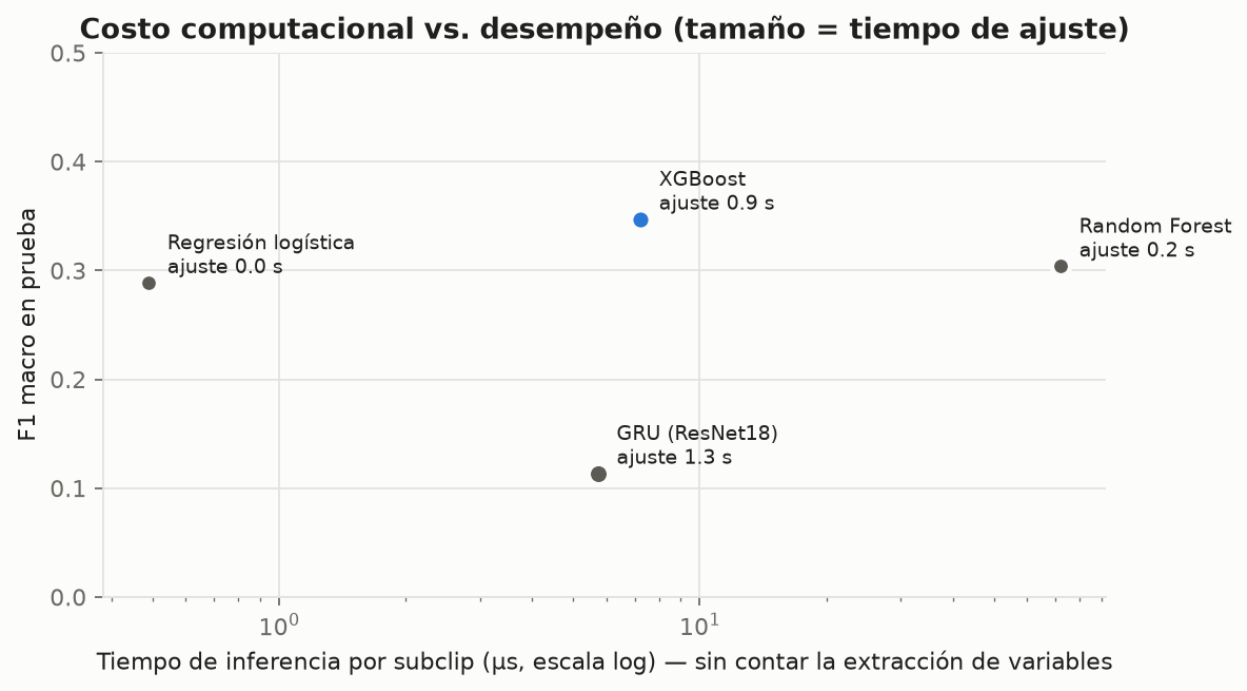
Las 15 variables más importantes de XGBoost (caída del F1 macro al permutarlas).



En costo computacional (Figura 8), los cuatro modelos clasifican un subclip en microsegundos y se ajustan en menos de 2 s una vez elegidos los hiperparámetros; el costo real está en la extracción de variables (flujo óptico, YOLO y ResNet18), común a todos. Por eso el criterio de selección es el desempeño, no el tiempo.

Figura 8

Tiempo de inferencia frente a F1 macro (el tamaño del punto es el tiempo de ajuste).



Modelo seleccionado. Se elige **XGBoost** como modelo principal: mejor F1 macro, mejor F1 en pase (la clase de interés del reto) y la estimación de validación más estable. La aplicación muestra además los otros tres modelos y su promedio para que el usuario compare sus respuestas.

4.6 Aplicación

Se construyó una aplicación web con **Streamlit** (app/app.py), elegido porque permite escribir la interfaz en el mismo Python que el resto del proyecto y reutilizar directamente el preprocesamiento y los modelos. Las gráficas interactivas usan **Plotly** (zoom, información al pasar el cursor, leyendas que se pueden activar y desactivar). Se ejecuta con streamlit run app/app.py y tiene cuatro páginas:

10. **Resumen.** Cuenta la historia del proyecto en cinco pasos (datos, preprocesamiento, hallazgos del análisis exploratorio, modelos y resultado) junto a las gráficas que respaldan cada hallazgo.

11. **Explorar datos.** Las 762 ventanas tras la limpieza con las 48 variables de los modelos: filtros por conjunto, clase y partido; distribución de una variable por clase (caja, histograma o violín), dispersión entre dos variables, comparación por partido, matriz de correlación, tabla descargable y diccionario de variables.

12. **Clasificar video.** El usuario sube un video o elige uno de los cuatro ejemplos del partido de prueba, y decide si usar **todos los modelos o solo algunos**. Puede clasificar el momento del evento, un segundo concreto o **recorrer todo el video** con una ventana deslizante, que produce una línea de tiempo de probabilidades por clase. El recorte, el muestreo de cuadros y la extracción de variables ocurren sin intervención del usuario (src/inference.py). Se comprobó que la aplicación reproduce el entrenamiento: en los cuatro ejemplos las 16 predicciones (4 modelos × 4 videos) coinciden con las de la evaluación, y en tres de ellos todas las variables difieren menos de 0.3 % de las calculadas para entrenar.

13. **Rendimiento.** Métricas en el partido de prueba con gráficas interactivas: comparación de F1, métricas por clase, matriz de confusión (con o sin normalizar), curvas de precisión-recall, importancia de variables, curvas de ajuste y costo frente a desempeño. Un interruptor permite **ocultar toda esta información** y un selector elige qué modelos comparar.

Los colores siguen la teoría del color vista en clase. Cada clase tiene un color fijo en todas las gráficas de la aplicación y del informe: pase azul, disputa naranja y saque de banda verde azulado. Los tres se validaron para que se distingan entre sí con los tres tipos de daltonismo. El fondo, que significa "sin acción", va en gris neutro. Los modelos no tienen color

propio: se identifican por nombre y solo el mejor se resalta en azul, para no competir con los colores de clase. Las magnitudes (matrices de confusión, F1 por clase) usan una escala secuencial de un solo tono, de claro a oscuro, y las correlaciones una escala divergente rojo–azul con gris en el cero.

Figura 9

Página de resumen: la historia del proyecto con sus hallazgos.

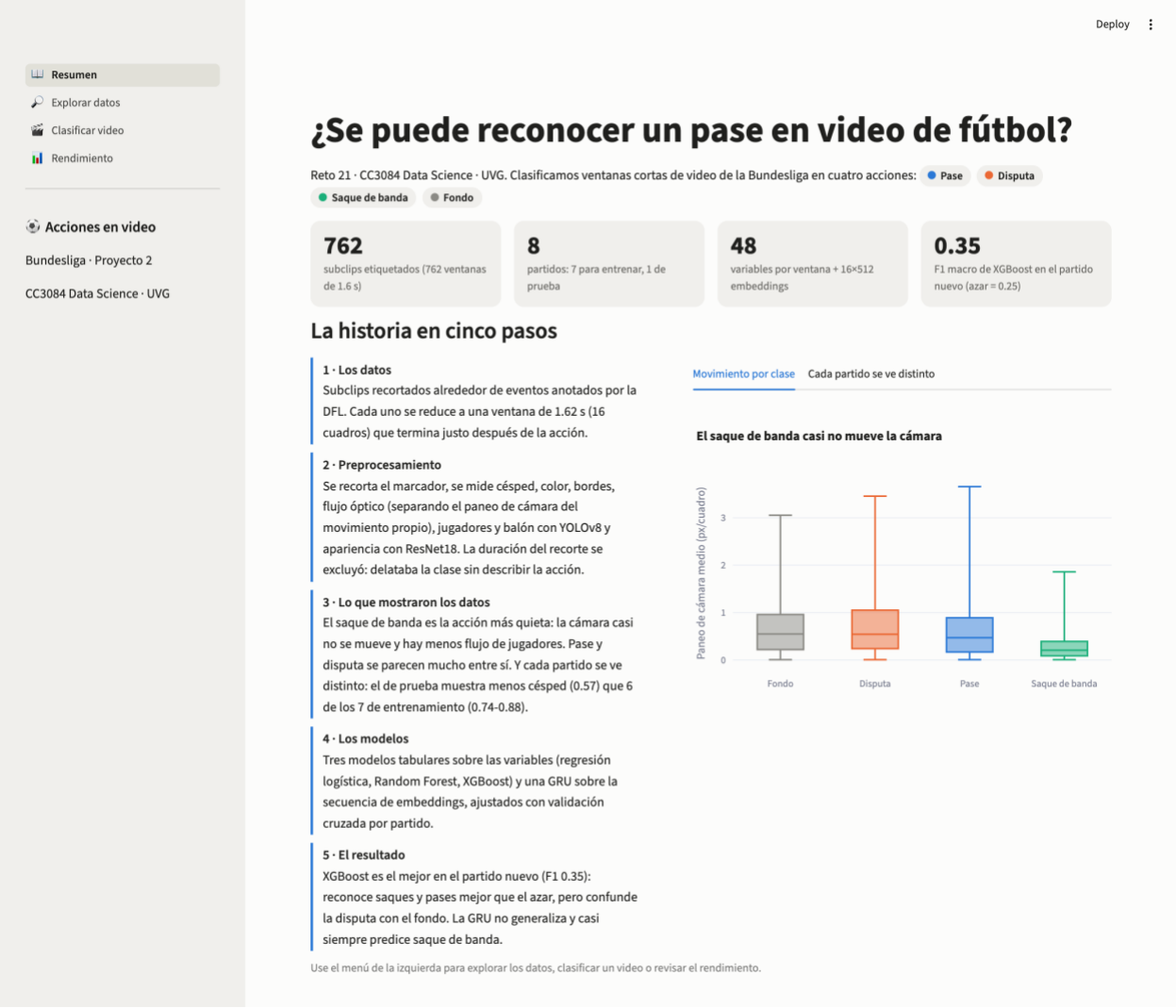


Figura 10

Exploración de datos: dispersión entre dos variables por clase y conjunto.

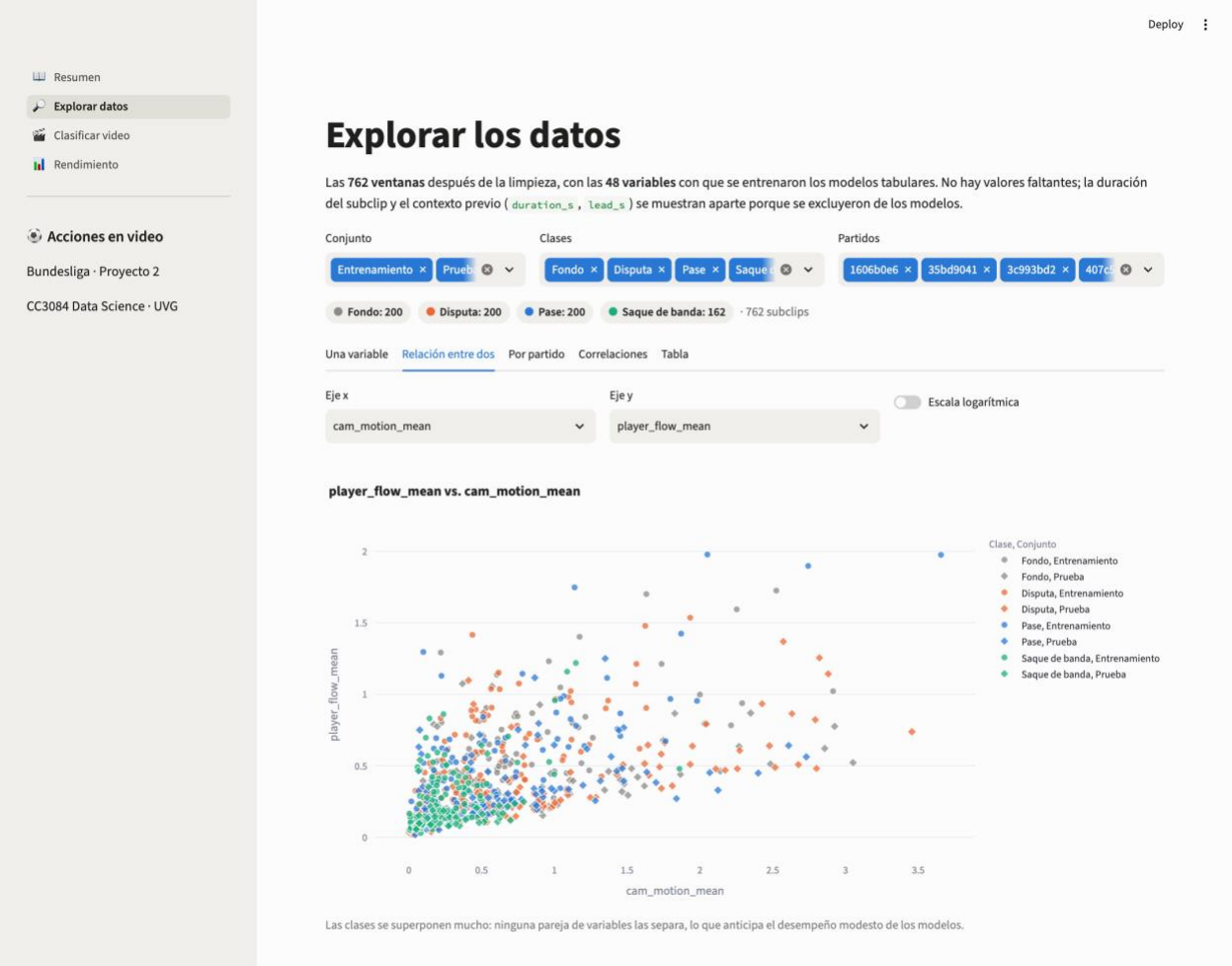


Figura 11

Clasificación de un video de ejemplo con los cuatro modelos y su promedio.

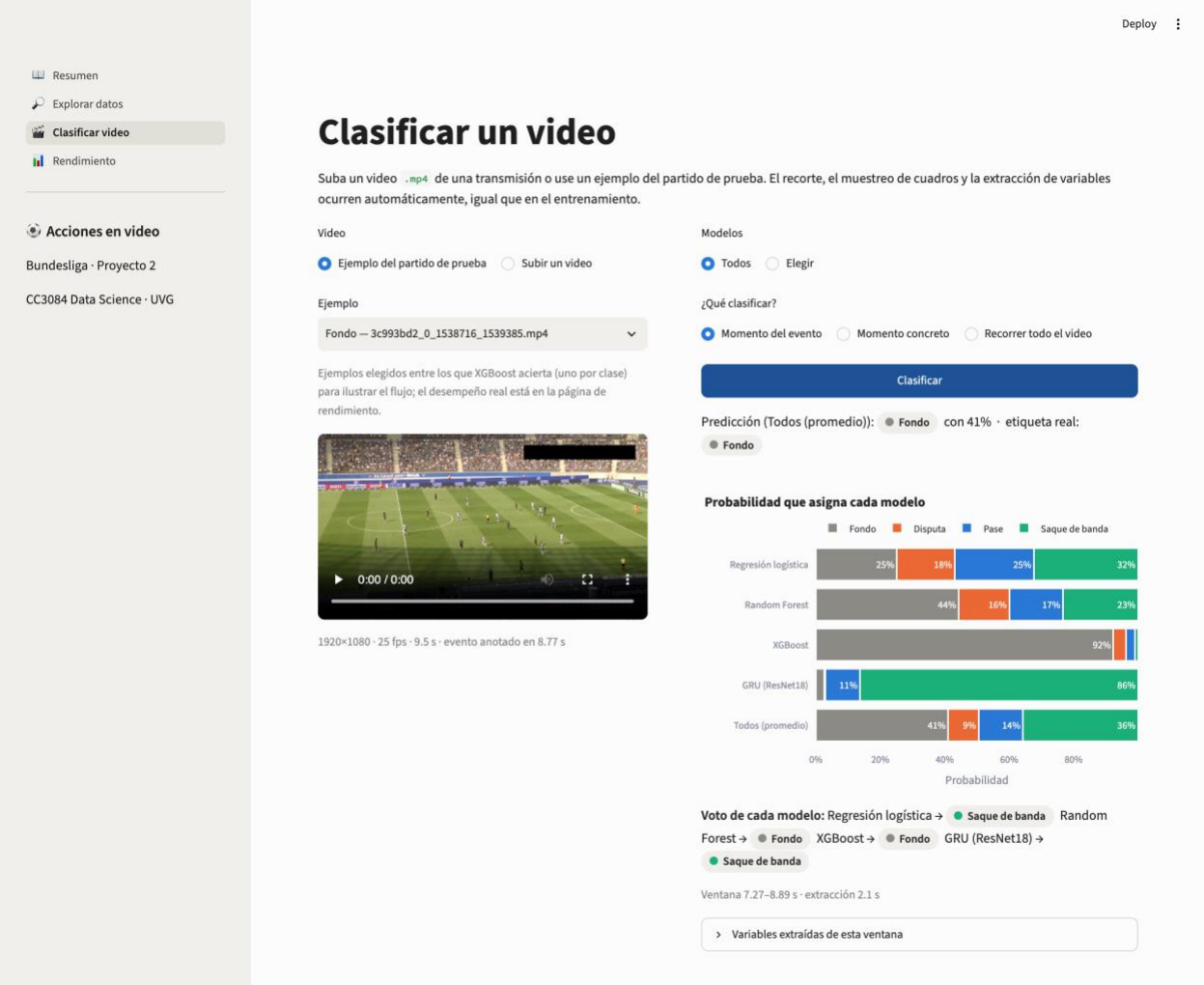
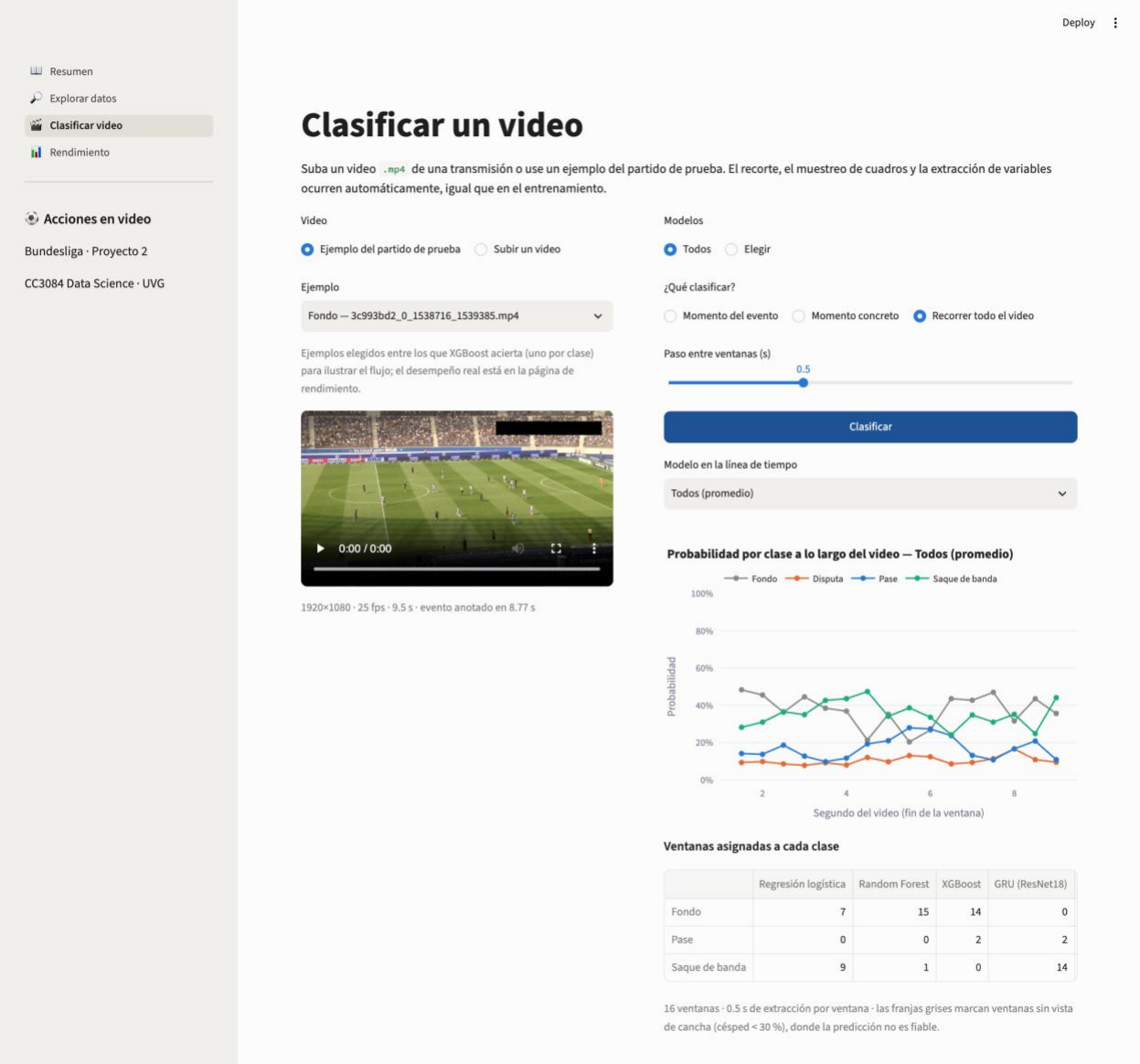


Figura 12

Recorrido de un video completo: probabilidad por clase a lo largo del tiempo.



	Regresión logística	Random Forest	XGBoost	GRU (ResNet18)
Fondo	7	15	14	0
Pase	0	0	2	2
Saque de banda	9	1	0	14

Figura 13

Página de rendimiento con las métricas visibles.

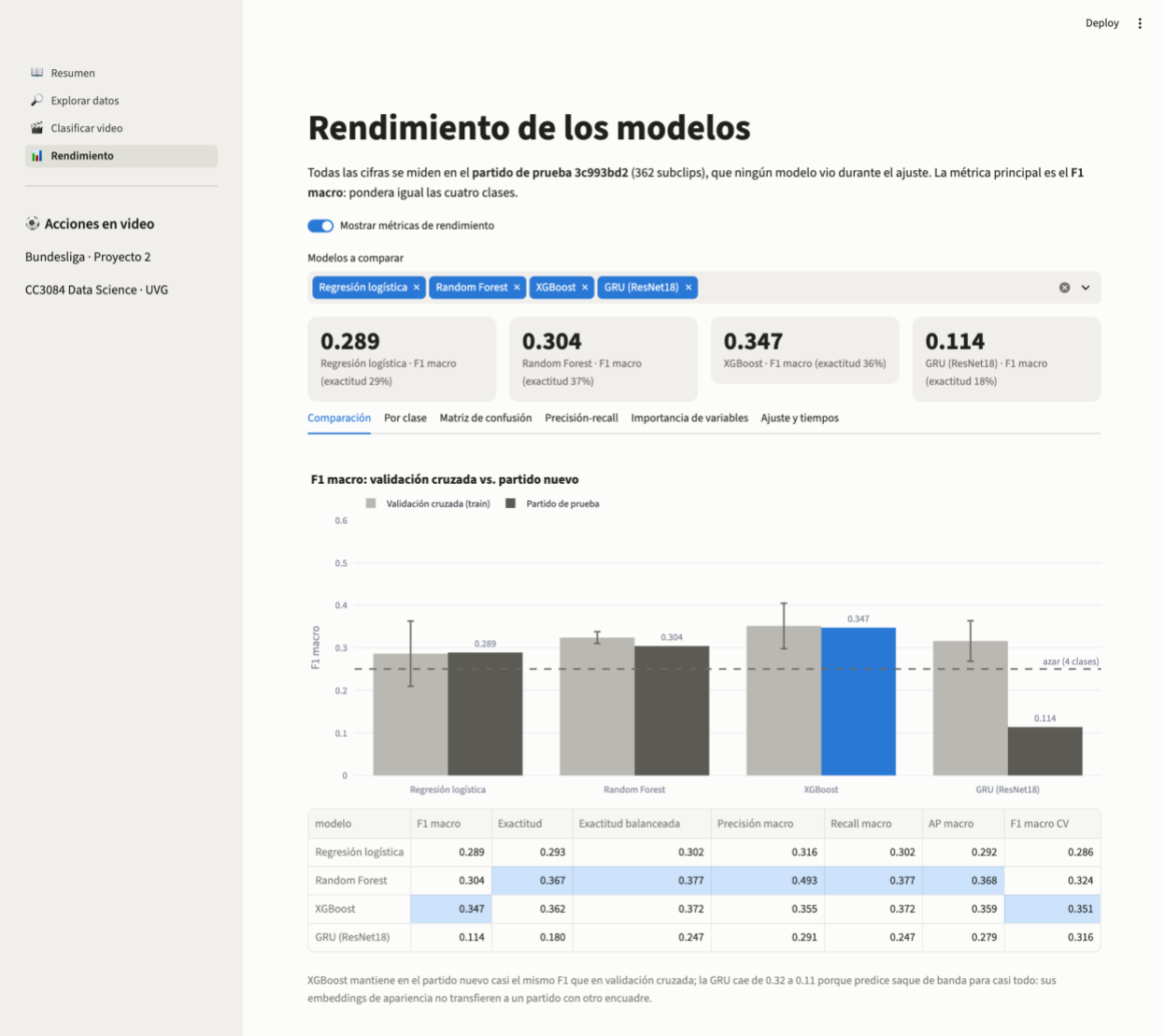


Figura 14

Página de rendimiento con las métricas ocultas por el usuario.



5. Conclusiones

5.1 Cumplimiento de los objetivos

Objetivo 1 — modelos tabulares. Se cumplió. Se entrenaron regresión logística, Random Forest y XGBoost sobre 48 variables por ventana, con búsqueda en malla y validación cruzada agrupada por partido. En el proceso se detectó y corrigió una fuga de información: la duración del subclip delataba la clase. Excluirla bajó el F1 de XGBoost de 0.381 a 0.347, pero deja cifras que sí corresponden a reconocer la acción y que se pueden reproducir en un video nuevo.

Objetivo 2 — modelo temporal y comparación. Se cumplió. La GRU sobre los *embeddings* de ResNet18 alcanzó 0.316 de F1 macro en validación cruzada, comparable a los tabulares, pero cayó a 0.114 en el partido de prueba porque predice saque de banda para casi todo. Con 400 secuencias, una representación de apariencia congelada no separa la acción del aspecto visual del partido.

Objetivo 3 — evaluación en un partido no visto. Se cumplió. Los cuatro modelos se evaluaron una sola vez en el partido 3c993bd2 con F1 macro, F1 por clase, matrices de confusión, curvas de precisión-recall, importancia por permutación y tiempos. **XGBoost es el**

mejor modelo (F1 macro 0.347, exactitud 36 %, frente a 0.25 del azar) y el único cuya validación cruzada anticipó bien su desempeño en prueba.

Objetivo 4 — aplicación. Se cumplió. La aplicación en Streamlit permite explorar las variables tras la limpieza, clasificar un video subido con uno, varios o todos los modelos (un momento o el video completo) y revisar el rendimiento en gráficas interactivas que se pueden ocultar. Se verificó que su preprocesamiento reproduce el del entrenamiento.

5.2 Efectividad de los algoritmos

Ningún modelo alcanza un desempeño suficiente para detectar de forma automática y confiable todas las acciones de un partido nuevo: el mejor F1 macro (0.347) está por encima del azar, pero lejos de un sistema utilizable sin supervisión. Los resultados sí muestran qué funciona y qué no:

- **Las variables de movimiento diseñadas a partir del análisis exploratorio son las que m**

aportan. **XGBoost usa sobre todo el flujo óptico propio de los jugadores (sin el paneo de cámara), la dispersión de jugadores detectados por YOLO y el movimiento de cámara. Con ellas reconoce el** pase **(F1 0.45, AP 0.48 frente a 0.28 del azar) y el** saque de banda** (F1 0.44). Para el pase, cuando está muy seguro acierta cerca del 90 % de las veces, así que es útil para encontrar los pases más claros.

- **La disputa no se distingue del juego sin acción** en una ventana de 1.6 s (F1 de 0.02 a

0.23 según el modelo). Separarla probablemente requiere seguir a los jugadores y el balón cuadro a cuadro.

- **El cambio entre partidos es el principal límite.** Los árboles alcanzan F1 = 1 en

entrenamiento y unos 0.33 en validación por partido. Con solo siete partidos para entrenar, el modelo no ve suficiente variedad de encuadres, iluminación y uniformes. Agregar complejidad no ayuda; agregar partidos sí debería.

- **Validar por partido fue clave.** Mezclar ventanas de un mismo encuentro entre entrenamiento y prueba habría producido cifras mucho más altas y engañosas. La validación cruzada agrupada anticipó bien el resultado de XGBoost en el partido nuevo (0.351 frente a 0.347).

5.3 Trabajo futuro

14. **Más partidos de entrenamiento** y normalización de color por partido para reducir el cambio de dominio, la limitación dominante.
15. **Ajuste fino de la CNN** o un modelo de video preentrenado en acciones, en lugar de *embeddings* de ImageNet congelados, con aumento de datos de color y encuadre para que la red temporal no dependa de la apariencia del partido.
16. **Seguimiento de jugadores y balón** entre cuadros (por ejemplo, YOLO reentrenado para el balón más un *tracker*) para describir la interacción que define una disputa o un pase.
17. **Localización temporal**: usar la línea de tiempo de la aplicación como detector de candidatos y evaluarla con tolerancia temporal sobre partidos completos.

6. Referencias

Akan, S., & Varlı, S. (2023). Use of deep learning in soccer videos analysis: Survey. *Multimedia Systems, 29*(3), 897–915. https://doi.org/10.1007/s00530-022-01027-0

Breiman, L. (2001). Random forests. *Machine Learning, 45*(1), 5–32. https://doi.org/10.1023/A:1010933404324

Chen, T., & Guestrin, C. (2016). XGBoost: A scalable tree boosting system. En *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining* (pp. 785–794). Association for Computing Machinery. https://doi.org/10.1145/2939672.2939785

Cho, K., van Merriënboer, B., Gulcehre, C., Bahdanau, D., Bougares, F., Schwenk, H., & Bengio, Y. (2014). Learning phrase representations using RNN encoder–decoder for statistical machine translation. En *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)* (pp. 1724–1734). Association for Computational Linguistics. https://doi.org/10.3115/v1/D14-1179

Friedman, J. H. (2001). Greedy function approximation: A gradient boosting machine. *The Annals of Statistics, 29*(5), 1189–1232. https://doi.org/10.1214/aos/1013203451

He, K., Zhang, X., Ren, S., & Sun, J. (2016). Deep residual learning for image recognition. En *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)* (pp. 770–778). IEEE. https://doi.org/10.1109/CVPR.2016.90

Saito, T., & Rehmsmeier, M. (2015). The precision-recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets. *PLoS ONE, 10*(3), Artículo e0118432. https://doi.org/10.1371/journal.pone.0118432

Torralba, A., & Efros, A. A. (2011). Unbiased look at dataset bias. En *2011 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)* (pp. 1521–1528). IEEE. https://doi.org/10.1109/CVPR.2011.5995347

Ullah, A., Ahmad, J., Muhammad, K., Sajjad, M., & Baik, S. W. (2018). Action recognition in video sequences using deep bi-directional LSTM with CNN features. *IEEE Access, 6*, 1155–1166. https://doi.org/10.1109/ACCESS.2017.2778011

Vrigkas, M., Nikou, C., & Kakadiaris, I. A. (2015). A review of human activity recognition methods. *Frontiers in Robotics and AI, 2*, Artículo 28. https://doi.org/10.3389/frobt.2015.00028